

It's time to now move on to the next part of the entire procedure in object detection, and we'll first begin by reviewing what will be done and what exactly we're looking for, from the program.

Like any standard object detection program, we will first insert the code for all the essential prerequisites and some additional tools that will be used to better enhance the learning algorithm in the Jupyter/Python environment. So without further ado, let's get started.

CODE STEPS

1. The downloaded images will all be resized to a certain dimension which will be then fed to the neural network framework. This will be done after the entire packages and libraries have been initialised.
2. Using the Object Detection API, bounding boxes will be set all around the object of interest in the images followed by a separate code that helps in detecting the objects and reporting the likeness by a percentage function.
3. The algorithm for detection will be extracted from a pre-rendered setup on the web that is openly available in the Google Developers mainframe and can be referenced easily.
4. If perhaps, you're running a typical setup with an Intel optimised network then getting some help through the VTune™ amplifier may be a good idea. While it may not be required for actual setups, VTune™ can help dish out sequential and multithreaded applications. Think of it as a status report generator that lets you know which algorithm takes up more computing space and cycle speeds.
5. It can even help you uncover CPU needs for specific portions of the algorithm and create a schema from least to most important or demanding. Let the system handle excessive workloads by generating Trace Event Formats which create a heat map or rather a preference map for those running on limited space.

```
• #! /bin/sh
• if [ "$#" -ne 1 ]; then
•     echo "Usage: collect.sh json_dir"
•     exit 1
• fi
• JSON_FILES=$1/*.json
• case "$SAMPLEX_E_COLLECT_CMD" in
• "start")
•     rm -rf $JSON_FILES
```